



中山大學
SUN YAT-SEN UNIVERSITY

本科生实验报告

实验课程: 操作系统原理实验

任课教师: 刘宁

实验题目: 编译内核利用已有内核构建 OS

专业名称: 计算机科学与技术 (人工智能与大数据方向)

学生姓名: 陶昕璨

学生学号: 24325242

实验地点: 实验中心 D501

实验时间: 2026/3/2-2026/3/5

目录

一、实验概述.....	1
二、预备知识与实验环境.....	1
三、实验任务.....	1
四、实验步骤与实验结果.....	1
五、实验总结与心得体会.....	10
六、附录：参考资料清单.....	10

Section 1 实验概述

熟悉现有 Linux 内核的编译过程和启动过程，并在自行编译内核的基础上构建简单应用并启动。同时，会利用精简的 Busybox 工具集构建简单的 OS，熟悉现代操作系统的构建过程。此外，同学们会熟悉编译环境、相关工具集，并能够实现内核远程调试。

Section 2 预备知识与实验环境

- 预备知识：x86 汇编语言程序设计、Linux 系统命令行工具、内核/Shell/GDB 基础、C++程序运行
- 实验环境：
 - 虚拟机版本/处理器型号：openSUSE（64-bit）
 - 代码编辑环境：Vim（终端）/ Gedit（图形化）
 - 代码编译工具：GCC、qemu-system-i386、GDB
 - 重要三方库信息：libc6-dev-i386、gcc-multilib

Section 3 实验任务

- 实验任务 1：环境配置，包括代码编辑环境、编译环境、运行环境、调试环境等
- 实验任务 2：下载并编译 i386（32 位）内核
- 实验任务 3：并利用 qemu 启动内核并且用 gdb 进行调试
- 实验任务 4：熟悉制作 initramfs 的方法
- 实验任务 5：编译 i386 版本的 Busybox，随内核启动，构建简单的 OS

Section 4 实验步骤与实验结果

----- 实验任务 1 -----

- 任务要求：对虚拟机进行环境配置，包括代码编辑环境、编译环境、运行环境、调试环境等（配置 C/C++以及其他工具）
- 实验步骤：

①配置 C/C++环境：在终端输入如下命令

```
sudo apt install binutils
sudo apt install gcc
```

②安装其他工具便于后续实验

```
sudo apt install qemu
sudo apt install cmake
sudo apt install libncurses5-dev
sudo apt install bison
sudo apt install flex
sudo apt install libssl-dev
sudo apt install libc6-dev-i386
sudo apt install gcc-multilib
sudo apt install g++-multilib
```

③下载适配 ubuntu 18.04 的 nasm 2.15 版本

```
//下载源码
wget https://www.nasm.us/pub/nasm/releasebuilds/2.15.05/nasm-2.15.05.tar.bz2
//解压
tar xvf nasm-2.15.05.tar.bz2
//进入目录（注意必须要进入目录才能完成接下来的配置任务
cd nasm-2.15.05
//配置
./configure
//编译
make
//安装
sudo make install
```

④安装 vscode 以及在 vscode 中安装汇编、C/C++插件。vscode 将作为一个有力的代码编辑器

```
//在 Ubuntu 18.04 上，使用 Snap 可以很好地解决依赖问题：
//安装 Snap
sudo apt update
sudo apt install snapd
//安装 VS Code:
sudo snap install code --classic
//在终端输入 code 启动vscode

//安装汇编、C/C++插件
code --install-extension ms-vscode.cpptools
code --install-extension ajshort.nasm
```

● 实验结果展示：

①在终端输入 `gcc -v`，显示 `gcc` 版本号，安装成功

```
gcc version 7.5.0 (Ubuntu 7.5.0-3ubuntu1~18.04)
bigsun@bigsun-VirtualBox:~$
```

②成功安装 `nasm` 2.15 版本

```
bigsun@bigsun-VirtualBox:~/nasm-2.15.05$ nasm -v
NASM version 2.15.05 compiled on Mar 7 2026
```

----- 实验任务 2 -----

- 任务要求：下载 Linux 内核并进行编译
- 思路分析：主要分析如何得到预期结果，有哪些要注意的地方等。
- 实验步骤：

①创建 `lab1` 文件夹

```
bigsun@bigsun-VirtualBox:~$ mkdir ~/lab1
bigsun@bigsun-VirtualBox:~$ cd ~/lab1
bigsun@bigsun-VirtualBox:~/lab1$
```

②到 <https://www.kernel.org/> 下载内核 5.10(或者更新的内核版本)到文件夹 `~/lab1`

mainline:	7.0-rc2	2026-03-01	[tarball]	[patch]	[inc. patch]	[view diff]	[browse]
stable:	6.19.6	2026-03-04	[tarball]	[pgp]	[patch]	[inc. patch]	[view diff] [browse] [changelog]
longterm:	6.18.16	2026-03-04	[tarball]	[pgp]	[patch]	[inc. patch]	[view diff] [browse] [changelog]
longterm:	6.12.76	2026-03-05	[tarball]	[pgp]	[patch]	[inc. patch]	[view diff] [browse] [changelog]
longterm:	6.6.129	2026-03-05	[tarball]	[pgp]	[patch]	[inc. patch]	[view diff] [browse] [changelog]
longterm:	6.1.166	2026-03-05	[tarball]	[pgp]	[patch]	[inc. patch]	[view diff] [browse] [changelog]
longterm:	5.15.202	2026-03-04	[tarball]	[pgp]	[patch]	[inc. patch]	[view diff] [browse] [changelog]
longterm:	5.10.252	2026-03-04	[tarball]	[pgp]	[patch]	[inc. patch]	[view diff] [browse] [changelog]
linux-next:	next-20260306	2026-03-06					[browse]

③解压并进入

```
xz -d linux-5.10.252.tar.xz
tar -xvf linux-5.10.252.tar
cd linux-5.10.252
```

```
bigsun@bigsun-VirtualBox:~/lab1$ cd linux-5.10.252
bigsun@bigsun-VirtualBox:~/lab1/linux-5.10.252$
```

④将内核编译成 i386 32 位版本

```
make i386_defconfig
make menuconfig
```

进入图像界面后依次选择 `Kernel hacking`（内核调试）、`Compile-time checks and compiler options`（编译时检查何编译器选项），最后在[] `Compile the kernel with`

debug info（编译内核时包含调试信息）输入 Y 勾选，保存退出

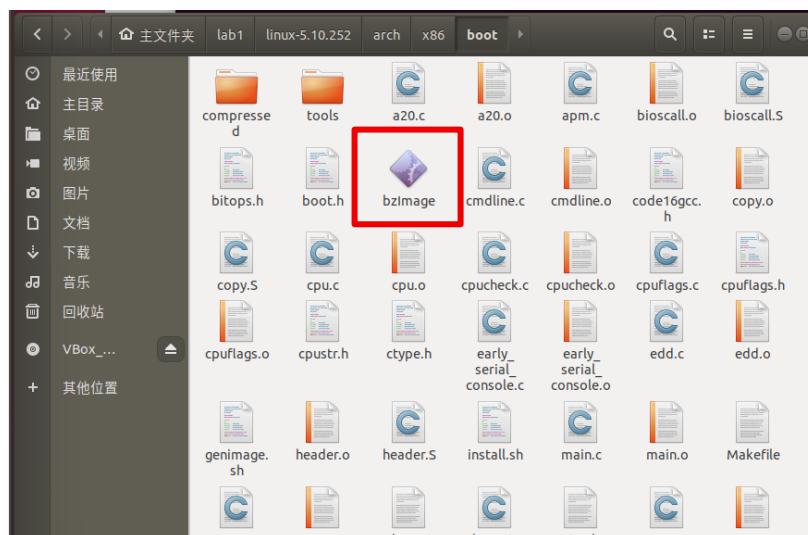
```
[*] Compile the kernel with debug info
[ ] Reduce debugging information (NEW)
[ ] Compressed debugging information (NEW)
[ ] Produce split debuginfo in .dwo files (NEW)
[ ] Generate dwarf4 debuginfo (NEW)
[ ] Generate BTF typeinfo (NEW)
[ ] Provide GDB scripts for kernel debugging (NEW)
[*] Enable __must_check logic
(1024) Warn for stack frames larger than
[ ] Strip assembler-generated symbols during link
[ ] Generate readable assembler code
[ ] Install uapi headers to usr/include
[ ] Enable full Section mismatch analysis
[*] Make section mismatch errors non-fatal
-*- Compile the kernel with frame pointers
[ ] Force weak per-cpu definitions
```

⑤编译内核

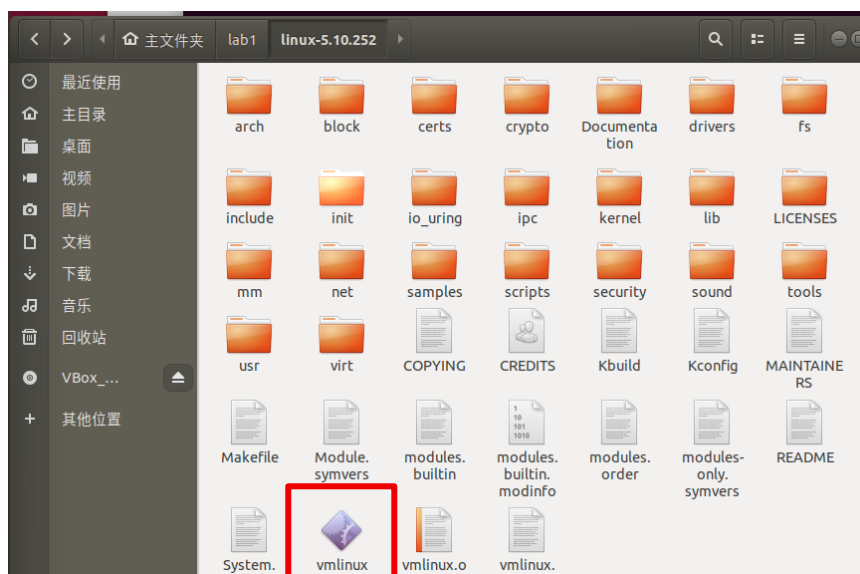
```
make -j8
```

● 实验结果展示：

①Linux 压缩镜像 linux-5.10.19/arch/x86/boot/bzImage



②符号表 linux-5.10.19/vmlinux



实验任务 3

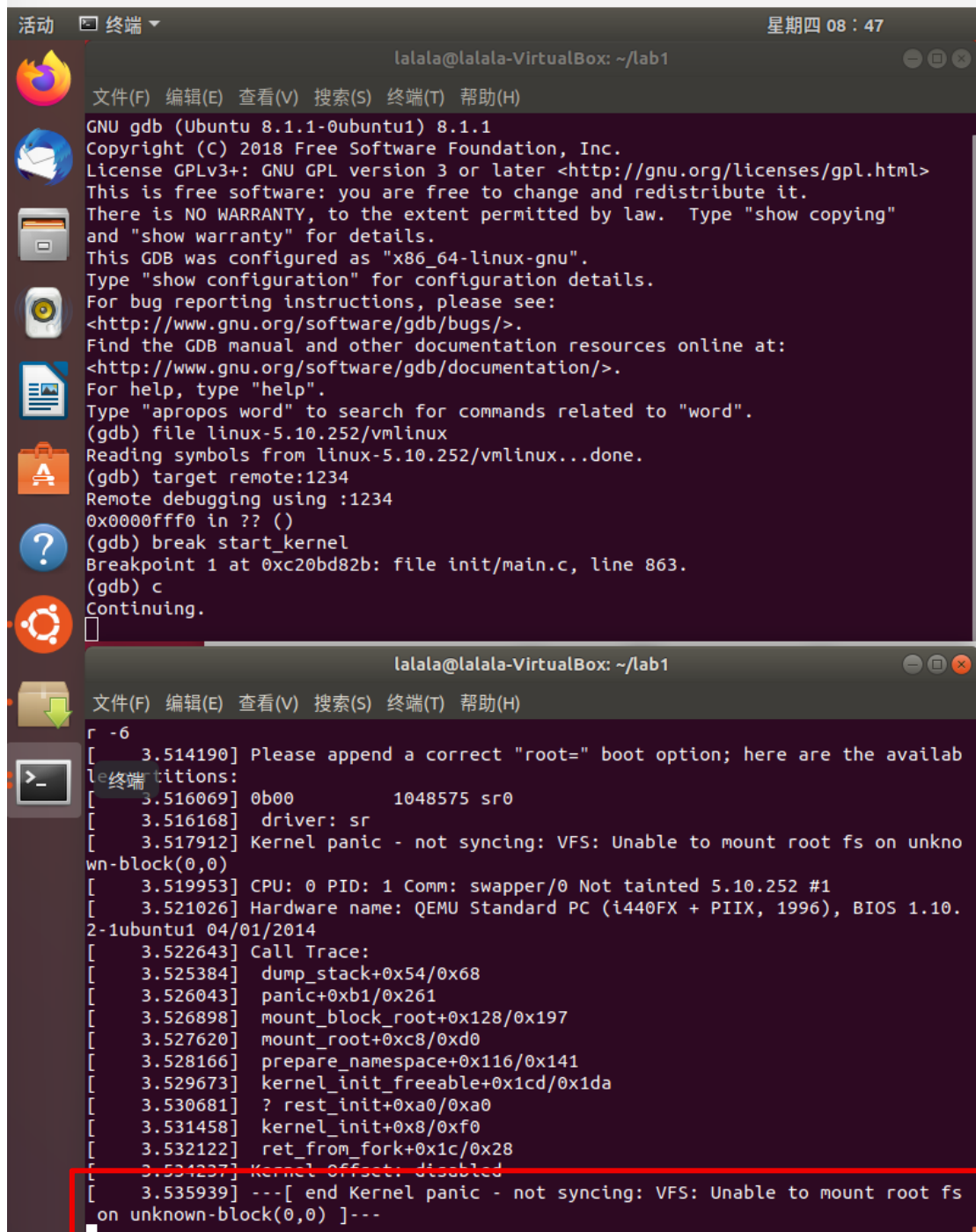
- 任务要求：使用 qemu 启动内核并使用 gdb 开启远程调试
- 思路分析：
 1. 全部过程需要在文件夹~/lab1 下进行（此处由于部分原因我创建了 lab2，并在 lab2 文件夹运行）
 2. 在另外一个 Terminal 下启动 gdb，注意，不要关闭 qemu 所在的 Terminal
- 实验步骤：
 - ①进入 lab1 文件夹，使用 qemu 启动内核并开启远程调试

```
//进入~/lab1
cd ~/lab1
//使用qemu启动内核并开启远程调试
qemu-system-i386 -kernel linux-5.10.19/arch/x86/boot/bzImage -s -S -append "console=ttyS0" -nographic
```

②gdb 调试

```
//启动gdb
gdb
//加载符号表
file linux-5.10.19/vmlinix
//连接已经启动的qemu进行调试。
target remote:1234
//为start_kernel函数设置断点。
break start_kernel
//运行
c
```

- 实验结果展示：通过执行前述代码，可得下图结果。



```
lalala@lalala-VirtualBox: ~/lab1
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
GNU gdb (Ubuntu 8.1.1-0ubuntu1) 8.1.1
Copyright (C) 2018 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word".
(gdb) file linux-5.10.252/vmlinux
Reading symbols from linux-5.10.252/vmlinux...done.
(gdb) target remote:1234
Remote debugging using :1234
0x0000ffff in ?? ()
(gdb) break start_kernel
Breakpoint 1 at 0xc20bd82b: file init/main.c, line 863.
(gdb) c
Continuing.
[0]

lalala@lalala-VirtualBox: ~/lab1
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
r -6
[ 3.514190] Please append a correct "root=" boot option; here are the availab
le终端 titions:
[ 3.516069] 0b00          1048575 sr0
[ 3.516168] driver: sr
[ 3.517912] Kernel panic - not syncing: VFS: Unable to mount root fs on unknow
n-block(0,0)
[ 3.519953] CPU: 0 PID: 1 Comm: swapper/0 Not tainted 5.10.252 #1
[ 3.521026] Hardware name: QEMU Standard PC (i440FX + PIIX, 1996), BIOS 1.10.
2-1ubuntu1 04/01/2014
[ 3.522643] Call Trace:
[ 3.525384] dump_stack+0x54/0x68
[ 3.526043] panic+0xb1/0x261
[ 3.526898] mount_block_root+0x128/0x197
[ 3.527620] mount_root+0xc8/0xd0
[ 3.528166] prepare_namespace+0x116/0x141
[ 3.529673] kernel_init_freeable+0x1cd/0x1da
[ 3.530681] ? rest_init+0xa0/0xa0
[ 3.531458] kernel_init+0x8/0xf0
[ 3.532122] ret_from_fork+0x1c/0x28
[ 3.534237] Kernel offset: disabled
[ 3.535939] ---[ end Kernel panic - not syncing: VFS: Unable to mount root fs
on unknown-block(0,0) ]---
```

实验任务 4

- 任务要求: 制作 Initramfs, 让系统 mount 上 initrd (init ram disk) 及其 initramfs 文件系统
- 实验步骤:
 - ①在 lab2 文件夹中编写 helloworld 程序并且进行编译成 32 位可执行文件

```
//打开~/lab2
cd ~/lab2

//使用echo命令快速创建helloworld程序代码
echo '#include <stdio.h>

void main()
{
    printf("lab1: Hello World\n");
    fflush(stdout);
    /* 让程序打印完后继续维持在用户态 */
    while(1);
}' > helloworld.c

//编译成32位可执行文件
gcc -o helloworld -m32 -static helloworld.c
```

②用 cpio 打包 initramfs

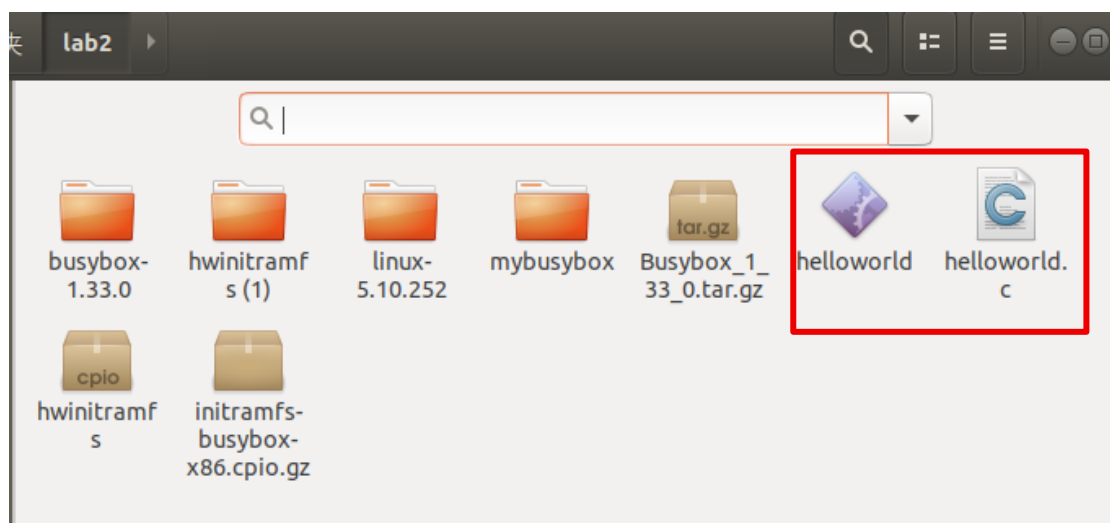
```
echo helloworld | cpio -o --format=newc > hwinitramfs
```

③启动内核，，并加载 initramfs

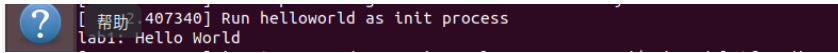
```
qemu-system-i386 -kernel linux-5.10.19/arch/x86/boot/bzImage -initrd hwinitramfs -s -S -
append "console=ttyS0 rdinit=helloworld" -nographic
```

● 实验结果展示：

①文件结构



②代码输出



----- 实验任务 5 -----

- 任务要求：下载、编译并启动 busybox（把几百个常用的 Linux/Unix 命令打包成的单个轻量级可执行文件）

- 思路分析：

1. 注意本实验任务全过程在 lab2 文件夹中进行

2. 在图形化配置阶段注意设置去掉括号，否则会出现如下问题

```
lalala@lalala-VirtualBox:~/lab2/busybox-1.33.0$ make install /bin/sh: 1: Syntax error: "("  
unexpected /bin/sh: 1: Syntax error: "(" unexpected
```

- 实验步骤：

①下载 Busybox 到 lab2

```
//使用wget命令下载  
wget https://busybox.net/downloads/busybox-1.33.0.tar  
//重命名  
mv busybox-1.33.0.tar.bz2 Busybox_1_33_0.tar.gz
```

②解压

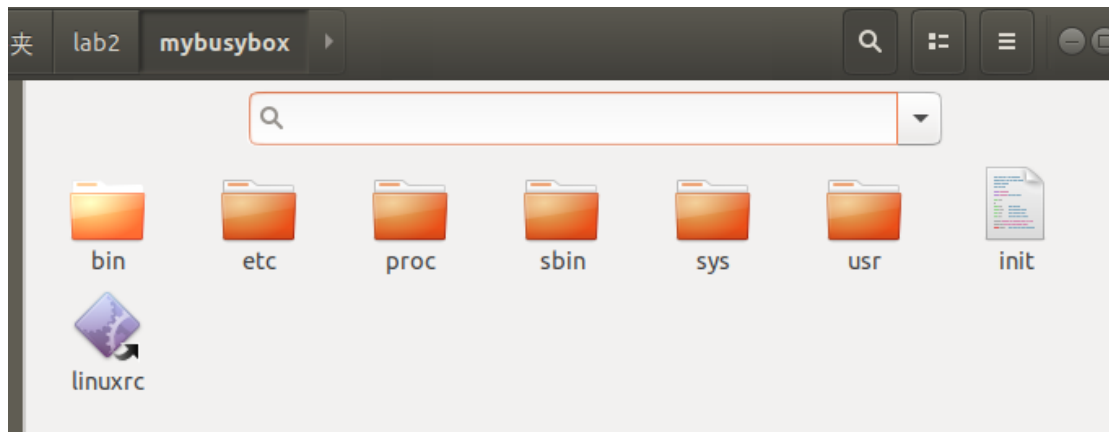
```
tar -xf Busybox_1_33_0.tar.gz
```

③编译并安装 busybox

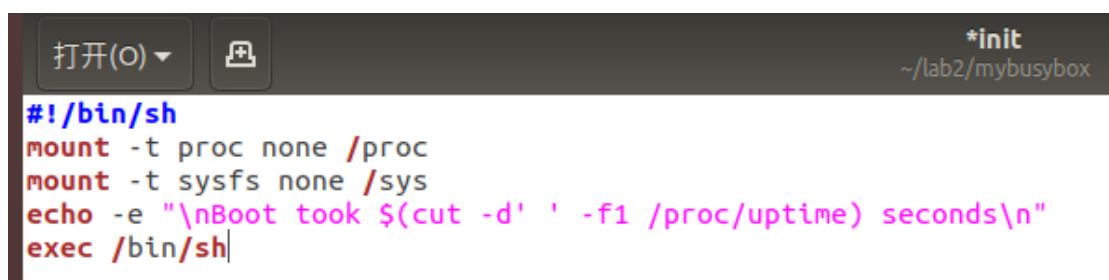
```
//进入 Busybox 源码目录  
cd busybox-1_33_0  
//初始化默认配置  
make defconfig  
//打开图形化配置界面  
make menuconfig  
//找到Build BusyBox as a static binary (no shared libs)，按Y勾选（括号出现*表示勾选成功）；  
找到() Additional CFLAGS，按回车，输入-m32 -march=i386，回车确认；  
找到() Additional LDFLAGS，按回车，输入-m32，回车确认  
//编译（-j8 表示用 8 线程编译）  
make -j8  
//安装  
make install
```

④制作基于 Busybox 的 Initramfs

Step1：将安装在_install 目录下的文件和目录取出放在~/lab2/mybusybox 处

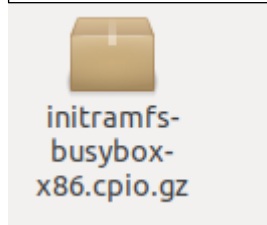


Step2: 制作 init 程序并给 init 脚本添加执行权限



Step3: 打包生成 Initramfs 镜像

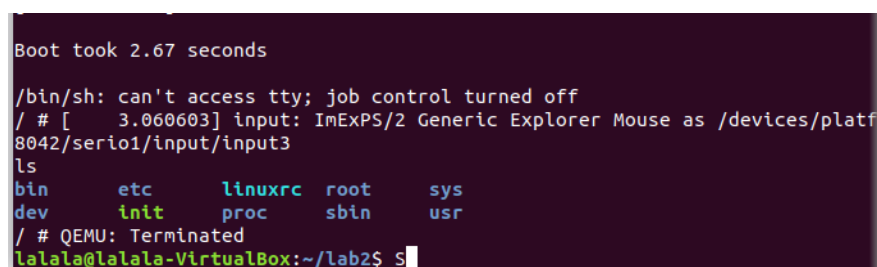
```
find . -print0 | cpio --null -ov --format=newc | gzip -9 > ~/lab1/initramfs-busybox-x86.cpio.gz
```



⑤加载 busybox

```
cd ~/lab1
qemu-system-i386 -kernel linux-5.10.19/arch/x86/boot/bzImage -initrd initramfs-busybox-x86.cpio.gz
-nographic -append "console=ttyS0"
```

● 实验结果展示:



Section 5 实验总结与心得体会

1. 在虚拟机设置时需要分配足够大的硬盘，配置复制粘贴以及拖拽等功能来提高效率，学会了如何来调整屏幕大小，为实验做准备
2. 编译调试时需要进入根目录，否则会显示

```
[ end Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0) ]---
```

3. 通过本次实验我基本掌握了 Linux 内核调试、Busybox 编译及如何制作 Initramfs，并且了解了 Initramfs 的作用，理清了极简 Linux 系统“内核+根文件系统”的构成逻辑，在本次实验后着手写报告本来想再重新做一次结果电脑没内存了，所有前面实验截图 username 和后面有差异

Section 6 附录：参考资料清单

实验指导——[SYSU-2023-Spring-Operating-System: 中山大学2023学年春季操作系统课程 - Gitee.com](#)